

PROOFS ARE PROGRAMS

Lecture 05 - Sebastian Holler

**POLYMORPHISM, HIGHER-ORDER-FUNCTIONS AND
MORE**

BACK TO LISTS

```
Inductive natlist : Type :=  
| nat_nil  
| nat_cons (n : nat) (l : natlist).
```

BACK TO LISTS

```
Inductive natlist : Type :=  
  | nat_nil  
  | nat_cons (n : nat) (l : natlist).
```

```
Definition mylist :=  
  nat_cons 1 (nat_cons 2 (nat_cons 3 nat_nil)).  
Definition mylist1 := 1 :: (2 :: (3 :: nat_nil)).  
Definition mylist2 := 1 :: 2 :: 3 :: nat_nil.  
Definition mylist3 := [1;2;3].
```

BACK TO LISTS

```
Inductive natlist : Type :=  
  | nat_nil  
  | nat_cons (n : nat) (l : natlist).
```

```
Definition mylist :=  
  nat_cons 1 (nat_cons 2 (nat_cons 3 nat_nil)).  
Definition mylist1 := 1 :: (2 :: (3 :: nat_nil)).  
Definition mylist2 := 1 :: 2 :: 3 :: nat_nil.  
Definition mylist3 := [1;2;3].
```

```
Fixpoint rev (l : natlist) : natlist :=  
  match l with  
  | nat_nil    => nat_nil  
  | h :: t     => rev t ++ [h]  
  end.
```

BACK TO LISTS

```
Inductive natlist : Type :=  
  | nat_nil  
  | nat_cons (n : nat) (l : natlist).
```

```
Definition mylist :=  
  nat_cons 1 (nat_cons 2 (nat_cons 3 nat_nil)).  
Definition mylist1 := 1 :: (2 :: (3 :: nat_nil)).  
Definition mylist2 := 1 :: 2 :: 3 :: nat_nil.  
Definition mylist3 := [1;2;3].
```

```
Fixpoint rev (l : natlist) : natlist :=  
  match l with  
  | nat_nil    => nat_nil  
  | h :: t    => rev t ++ [h]  
  end.
```

```
Theorem nil_app :  $\forall$  l : natlist,  
  [] ++ l = l.  
Proof. reflexivity. Qed.
```

List of Booleans

List of Booleans

```
Definition mylist := [true; true; false].
```

List of Booleans

```
Definition mylist := [true; true; false].
```

```
Inductive natlist : Type :=  
| nat_nil (* natlist *)  
| nat_cons (n : nat) (l : natlist).  
(* nat -> natlist -> natlist *)
```


List of Booleans

```
Definition mylist := [true; true; false].
```

```
Inductive natlist : Type :=  
| nat_nil (* natlist *)  
| nat_cons (n : nat) (l : natlist).  
(* nat -> natlist -> natlist *)
```

```
Inductive boollist : Type :=  
| bool_nil (* boollist *)  
| bool_cons (b : bool) (l : boollist).  
(* bool -> boollist -> boollist *)
```

Generalizing Lists

Generalizing Lists

```
1 list : Type -> Type
2
3 list (bool) -> boollist
4 list (nat) -> natlist
```

Generalizing Lists

```
1 list : Type -> Type
2
3 list (bool) -> boollist
4 list (nat) -> natlist
```

Generalizing Lists

```
1 list : Type -> Type
2
3 list (bool) -> boollist
4 list (nat) -> natlist
```

```
Inductive list : Type :=
| nil          (* list *)
| cons (x : X) (l : list).
              (* X -> list -> list *)
```

Generalizing Lists

```
1 list : Type -> Type
2
3 list (bool) -> boollist
4 list (nat) -> natlist
```

```
Inductive list : Type :=
| nil          (* list *)
| cons (x : X) (l : list).
              (* X -> list -> list *)
```

Where does "X" come from?

Should both lists have the same type?

Generalizing Lists

```
1 Inductive list : Type :=  
2   | nil          (* list *)  
3   | cons (x : X) (l : list).  
4           (* X -> list -> list *)
```

Generalizing Lists

```
1 Inductive list : Type :=  
2   | nil          (* list *)  
3   | cons (x : X) (l : list).  
4           (* X -> list -> list *)
```

```
1 Inductive list : Type :=  
2   | nil          (* list X *)  
3   | cons (x : X) (l : list X).  
4           (* X -> list X -> list X *)
```


Generalizing Lists

```
1 Inductive list : Type :=  
2   | nil          (* list *)  
3   | cons (x : X) (l : list).  
4           (* X -> list -> list *)
```

```
1 Inductive list : Type :=  
2   | nil          (* list X *)  
3   | cons (x : X) (l : list X).  
4           (* X -> list X -> list X *)
```

```
1 Inductive list : Type :=  
2   | nil          (* forall X : Type, list X *)  
3   | cons (x : X) (l : list X).  
4           (* forall X : Type, X -> list X -> list X *)
```

Generalizing Lists

```
1 Inductive list : Type :=  
2   | nil          (* list *)  
3   | cons (x : X) (l : list).  
4           (* X -> list -> list *)
```

```
1 Inductive list : Type :=  
2   | nil          (* list X *)  
3   | cons (x : X) (l : list X).  
4           (* X -> list X -> list X *)
```

```
1 Inductive list : Type :=  
2   | nil          (* forall X : Type, list X *)  
3   | cons (x : X) (l : list X).  
4           (* forall X : Type, X -> list X -> list X *)
```

```
1 Inductive list (X : Type) : Type :=  
2   | nil          (* forall X : Type, list X *)  
3   | cons (x : X) (l : list X).  
4           (* forall X : Type, X -> list X -> list X *)
```

```
Inductive list (X : Type) : Type :=  
| nil  
| cons (x : X) (l : list X).
```

```
Inductive list (X : Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

```
1 Check nil :  $\forall$  X : Type, list X.  
2  
3 Check cons :  $\forall$  X : Type, X -> list X -> list X.  
4  
5 Check (nil nat) : list nat.  
6  
7 Check (cons nat 3 (nil nat)) : list nat.
```

```
Inductive list (X : Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

```
1 Check nil :  $\forall$  X : Type, list X.  
2  
3 Check cons :  $\forall$  X : Type, X -> list X -> list X.  
4  
5 Check (nil nat) : list nat.  
6  
7 Check (cons nat 3 (nil nat)) : list nat.
```

```
Inductive list (X : Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

```
1 Check nil :  $\forall$  X : Type, list X.  
2  
3 Check cons :  $\forall$  X : Type, X -> list X -> list X.  
4  
5 Check (nil nat) : list nat.  
6  
7 Check (cons nat 3 (nil nat)) : list nat.
```

```
Inductive list (X : Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

```
1 Check nil :  $\forall$  X : Type, list X.  
2  
3 Check cons :  $\forall$  X : Type, X -> list X -> list X.  
4  
5 Check (nil nat) : list nat.  
6  
7 Check (cons nat 3 (nil nat)) : list nat.
```

```
Inductive list (X : Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

```
1 Check nil : ∀ X : Type, list X.  
2  
3 Check cons : ∀ X : Type, X -> list X -> list X.  
4  
5 Check (nil nat) : list nat.  
6  
7 Check (cons nat 3 (nil nat)) : list nat.
```

```
Check cons bool true (cons nat 3 (nil nat)). (* ??? *)
```



```
Inductive list (X : Type) : Type :=  
  | nil  
  | cons (x : X) (l : list X).
```

```
1 Check nil : ∀ X : Type, list X.  
2  
3 Check cons : ∀ X : Type, X -> list X -> list X.  
4  
5 Check (nil nat) : list nat.  
6  
7 Check (cons nat 3 (nil nat)) : list nat.
```

```
Check cons bool true (cons nat 3 (nil nat)). (* ??? *)
```

```
(* The term "cons nat 3 (nil nat)" has type "list nat"  
   while it is expected to have type "list bool". *)
```

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=  
  match count with  
  | 0 => nil X  
  | S count' => cons X x (repeat X x count')  
  end.
```

```
Example test_repeat1 :  
  repeat nat 4 2 = cons nat 4 (cons nat 4 (nil nat)).
```

```
Example test_repeat2 :  
  repeat bool false 1 = cons bool false (nil bool).
```

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=  
  match count with  
  | 0 => nil X  
  | S count' => cons X x (repeat X x count')  
  end.
```

```
Example test_repeat1 :  
  repeat nat 4 2 = cons nat 4 (cons nat 4 (nil nat)).
```

```
Example test_repeat2 :  
  repeat bool false 1 = cons bool false (nil bool).
```

```
Check repeat. (* ??? *)
```

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=  
  match count with  
  | 0 => nil X  
  | S count' => cons X x (repeat X x count')  
end.
```

```
Example test_repeat1 :  
  repeat nat 4 2 = cons nat 4 (cons nat 4 (nil nat)).
```

```
Example test_repeat2 :  
  repeat bool false 1 = cons bool false (nil bool).
```

```
Check repeat. (* ??? *)
```

$\text{repeat} : \forall X : \text{Type}, X \rightarrow \text{nat} \rightarrow \text{list } X.$

TYPE ANNOTATION INFERENCE

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=
```

```
Fixpoint repeat' X x count : list X :=
```

.

TYPE ARGUMENT SYNTHESIS

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=  
  match count with  
  | 0 => nil X  
  | S count' => cons X x (repeat X x count')  
end.
```

```
Fixpoint repeat' X x count : list X :=  
  match count with  
  | 0 => nil _  
  | S count' => cons _ x (repeat' _ x count')  
end.
```

IMPLICIT ARGUMENTS

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=  
  match count with  
  | 0 => nil X  
  | s count' => cons X x (repeat X x count')  
  end.
```

```
Arguments nil {X}.
```

```
Arguments cons {X}.
```

```
Arguments repeat {X}.
```

```
(* ... *)
```

IMPLICIT ARGUMENTS

```
Fixpoint repeat (X : Type) (x : X) (count : nat) : list X :=  
  match count with  
  | 0 => nil X  
  | s count' => cons X x (repeat X x count')  
end.
```

```
Arguments nil {X}.
```

```
Arguments cons {X}.
```

```
Arguments repeat {X}.
```

```
Fixpoint repeat' {X : Type} (x : X) (count : nat) : list X  
:= (* same effect *)
```


EXPLICIT TYPE ARGUMENTS

```
Fail Definition mynil := nil.  
Definition mynil : list nat := nil. (* w/ type declaration *)  
Definition mynil' := @nil nat. (* disables implicit typing *)
```

STANDARD POLYMORPHIC LIST FUNCTIONS

```
Fixpoint rev {X:Type} (l:list X) : list X :=  
  match l with  
  | nil => nil  
  | cons h t => app (rev t) (cons h nil)  
end.
```

```
Fixpoint length {X : Type} (l : list X) : nat :=  
  match l with  
  | nil => 0  
  | cons _ l' => S (length l')  
end.
```

```
Example test_length1: length (cons 1 (cons 2 nil)) = 2.  
Proof. reflexivity. Qed.
```

HIGHER-ORDER-FUNCTIONS

Definition `inc (n : nat) : nat := S n.`

Definition `inc3times (n : nat) : nat :=
S (S (S n)).`

HIGHER-ORDER-FUNCTIONS

```
Definition inc (n : nat) : nat := S n.
```

```
Definition inc3times (n : nat) : nat :=  
  S (S (S n)).
```

```
1 Definition doit3times (f : nat -> nat) (n : nat) : nat :=  
2     f (f (f n)).  
3  
4 Check S : nat -> nat.  
5 Check inc : nat -> nat.
```

HIGHER-ORDER-FUNCTIONS

```
Definition inc (n : nat) : nat := S n.
```

```
Definition inc3times (n : nat) : nat :=  
  S (S (S n)).
```

```
1 Definition doit3times (f : nat -> nat) (n : nat) : nat :=  
2     f (f (f n)).  
3  
4 Check S : nat -> nat.  
5 Check inc : nat -> nat.
```

HIGHER-ORDER-FUNCTIONS

```
Definition inc (n : nat) : nat := S n.
```

```
Definition inc3times (n : nat) : nat :=  
  S (S (S n)).
```

```
1 Definition doit3times (f : nat -> nat) (n : nat) : nat :=  
2     f (f (f n)).  
3  
4 Check S : nat -> nat.  
5 Check inc : nat -> nat.
```

```
Definition doit3times {X : Type} (f : X->X) (n : X) : X :=  
  f (f (f n)).
```

```
Example test_doit3times :  
  doit3times inc 0 = inc3times 0.
```

ITER

ITER

```
Fixpoint iter {X : Type} (f : X->X) (n : nat) (s: X) : X :=  
  match n with  
  | 0      => s  
  | S n'   => f (iter f n' s)  
end.
```


ITER

```
Fixpoint iter {X : Type} (f : X->X) (n : nat) (s: X) : X :=  
  match n with  
  | 0      => s  
  | S n'  => f (iter f n' s)  
  end.
```

```
Example iter_test1:  
  iter S 3 0 = S (S (S 0)).
```

```
Example iter_test2:  
  iter inc 3 0 = S (S (S 0)).
```

ANONYMOUS FUNCTIONS

```
(fun n => n + 1) (fun n => n * n)
```

ANONYMOUS FUNCTIONS

`(fun n => n + 1) (fun n => n * n)`

```
Example iter_anon_1:  
  iter (fun n => n + 1) 3 0 = S (S (S 0)).
```

```
Example iter_anon_2:  
  iter (fun n => n * n) 5 0 = 0.
```

COLLECTION-ORIENTED PROGRAMMING

Intuition: Define general functions that operate on lists

List + Instructions $\rightarrow$ Output (e.g. new List)

FILTER

`list X`

`+`

`X -> bool`

`=>`

`list X`

FILTER

`[1; 2; 3; 42; 1337]`

`+`

`(fun x => even x)`

`=>`

`[2; 42]`

FILTER

```
Fixpoint filter {X:Type} (test: X->bool) (l:list X) :list X :=  
  match l with  
  | [] => []  
  | h :: t =>  
  
  end.
```

FILTER

```
Fixpoint filter {X:Type} (test: X->bool) (l:list X) :list X :=
  match l with
  | [] => []
  | h :: t =>
    if test h then h :: (filter test t)
    else filter test t
  end.
```


MAP

[1; 2; 3; 42; 1337]

+

(fun x => x + 1)

=>

[2; 3; 4; 43; 1338]

MAP

`[1; 2; 3; 42; 1337]`

`+`

`(fun x => even x)`

`=>`

`[false; true; false; true; false]`

MAP

```
Fixpoint map {X Y : Type} (f : X->Y) (l : list X) : list Y :=  
  match l with  
  | [] => []  
  | h :: t => (f h) :: (map f t)  
end.
```

MAP

```
Fixpoint map {X Y : Type} (f : X->Y) (l : list X) : list Y :=  
  match l with  
  | [] => []  
  | h :: t => (f h) :: (map f t)  
end.
```

```
Example test_map1: map (fun x => plus 3 x) [2;0;2] = [5;3;5].
```

FOLD

[1;2;3] --- sum ---> 6

[2;4;6] --- all_even ---> true

FOLD

`[1;2;3] --- sum ---> 6`

`[2;4;6] --- all_even ---> true`

`[1;2;3] --- plus_1 ---> [2;3;4]`

`[true; false; true] --- magic ---> [42; 42; 1337; 42; 42]`

FOLD

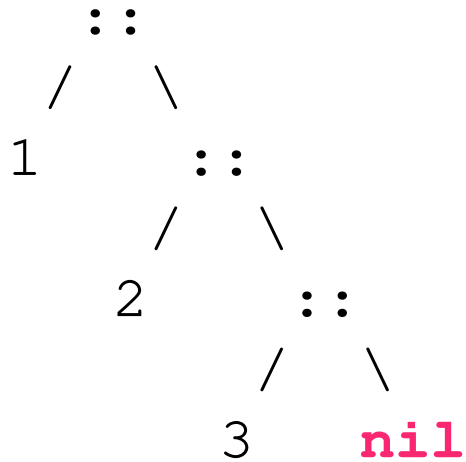
`[1;2;3] --- sum ---> 6`

`[2;4;6] --- all_even ---> true`

- Input list: `list X`
- Some function: `X -> ... -> Y`
- Return value: `Y`

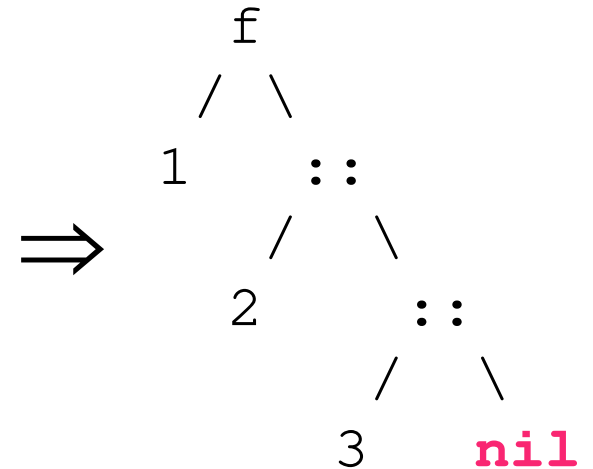
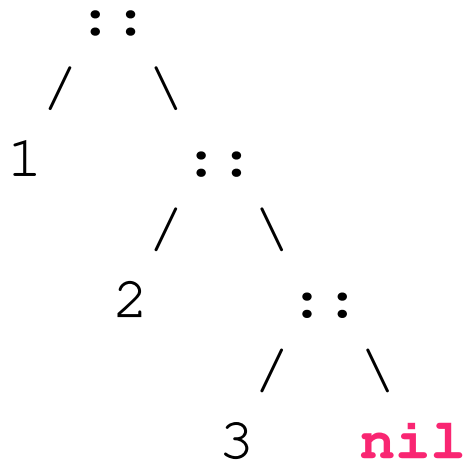
FOLD

`[1;2;3] = 1 :: (2 :: (3 :: nil))`



FOLD

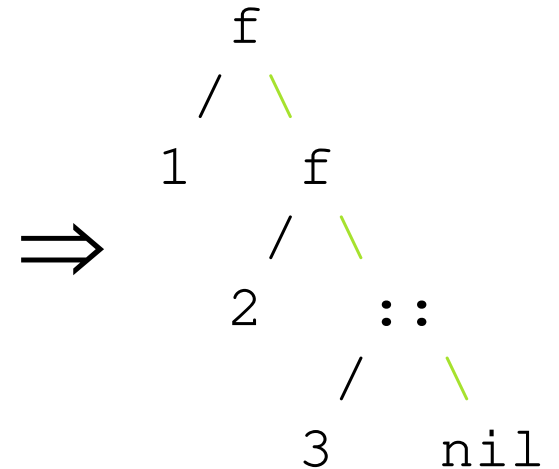
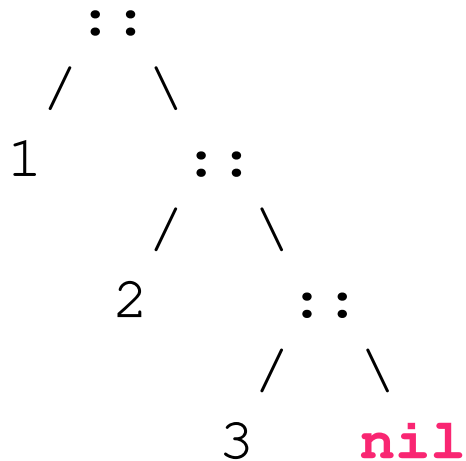
$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$



swap (::) $\rightarrow$ f

FOLD

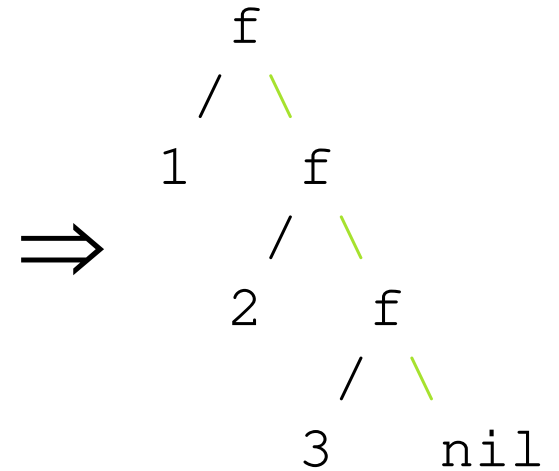
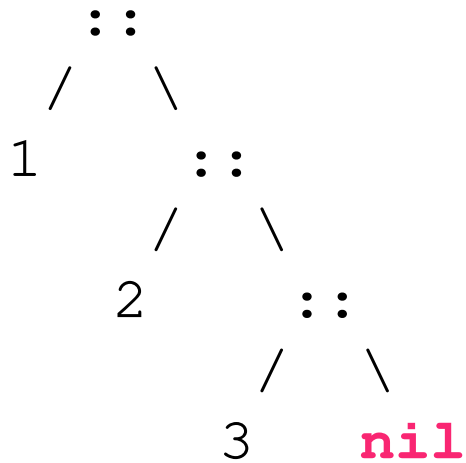
$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$



swap $(::) \rightarrow f$

FOLD

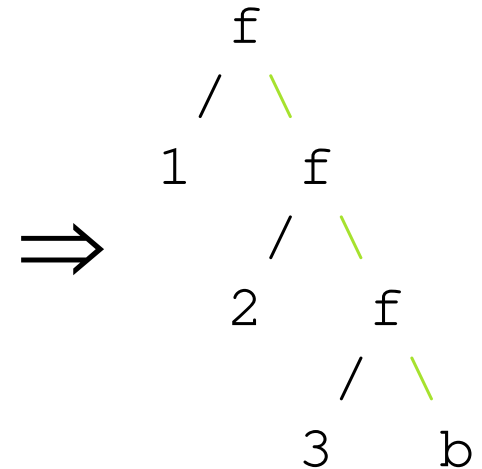
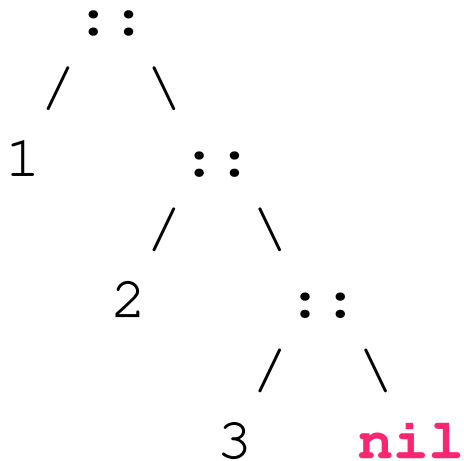
$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$



swap $(::) \rightarrow f$

FOLD

$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$

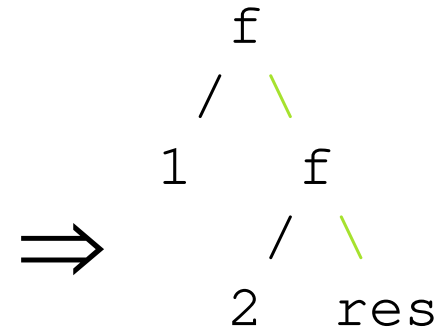
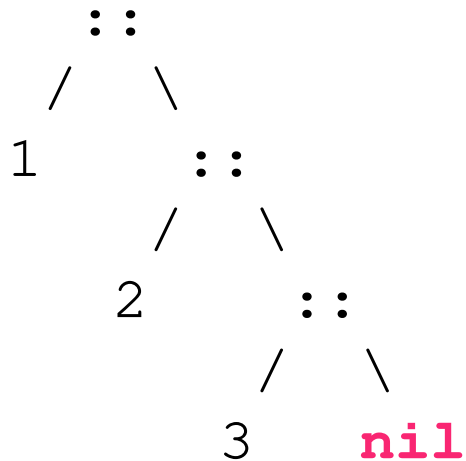


swap $(::) \rightarrow f$

$= f\ 1\ (f\ 2\ (f\ 3\ b))$

FOLD

$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$

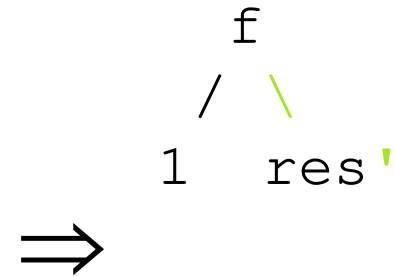
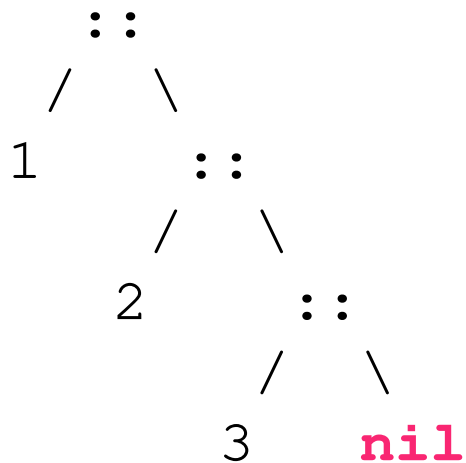


swap $(::) \rightarrow f$

$= f\ 1\ (f\ 2\ \text{res})$ where $\text{res} = f\ 3\ b$

FOLD

$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$

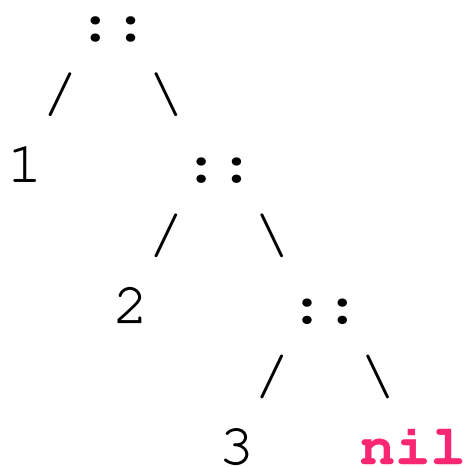


swap $(::) \rightarrow f$

$= f\ 1\ \text{res}'$ where $\text{res}' = f\ 2\ \text{res}$

FOLD

$[1;2;3] = 1 :: (2 :: (3 :: \text{nil}))$



res''



swap $(::) \rightarrow f$

$= \text{res}'$, where $\text{res}'' = f\ 1\ \text{res}'$

FOLD

`[1;2;3] --- sum ----> 6`

`[2;4;6] --- all_even ----> true`

- Input list: `list X`
- Some function: `X -> ... -> Y`
- Return value: `Y`
- Neutral element: `Y (for nil)`

FOLD

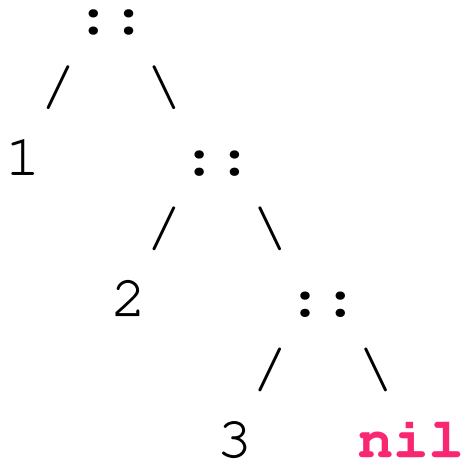
`[1;2;3] --- sum ----> 6`

`[2;4;6] --- all_even ----> true`

- Input list: `list X`
- Some function: `X -> Y -> Y`
- Return value: `Y`
- Neutral element: `Y (for nil)`

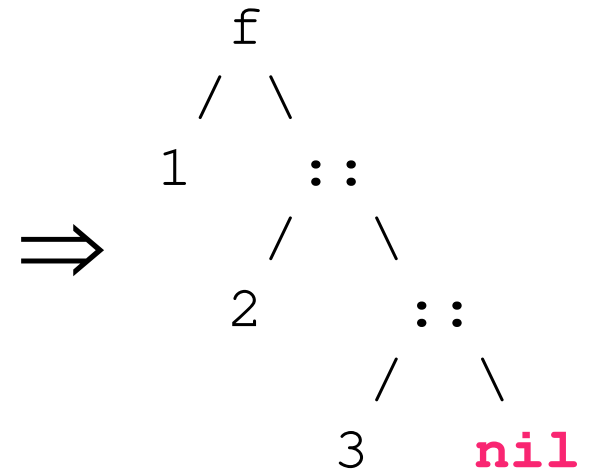
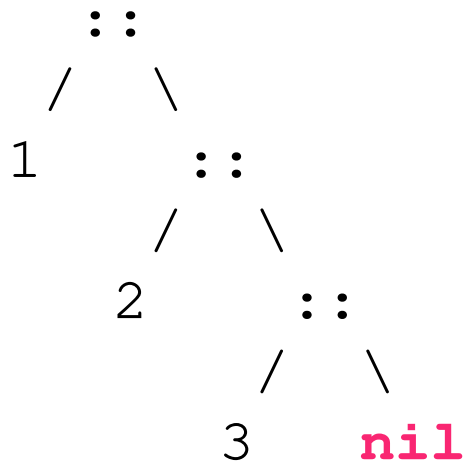
FOLD — SUM

[1;2;3] --- sum ----> 6 (f = (+), b = 0)



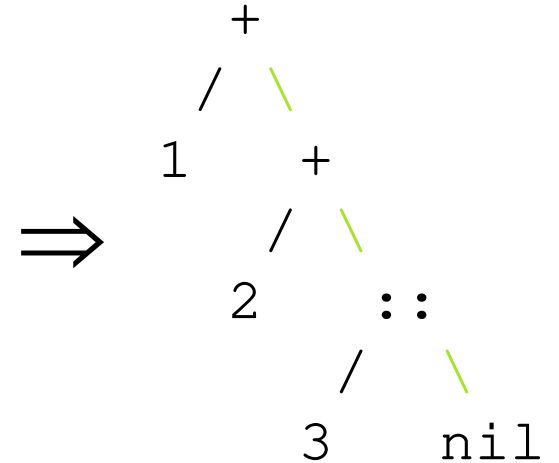
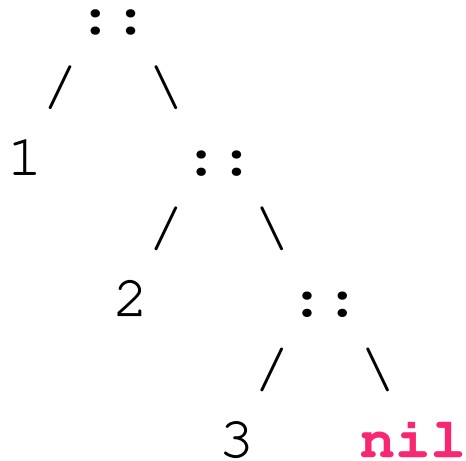
FOLD — SUM

[1;2;3] --- sum ----> 6 (f = (+), b = 0)



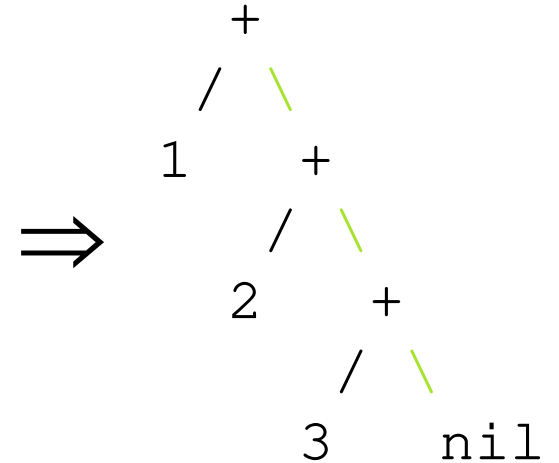
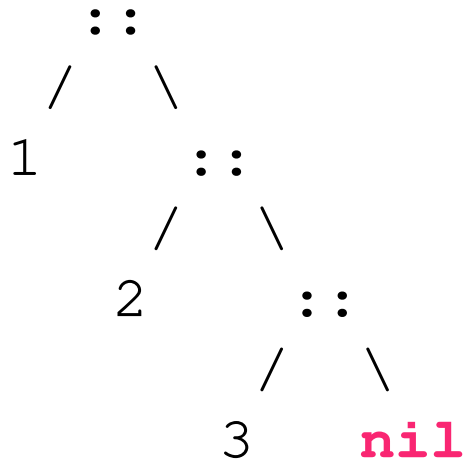
FOLD — SUM

[1;2;3] --- sum ----> 6 (f = (+), b = 0)



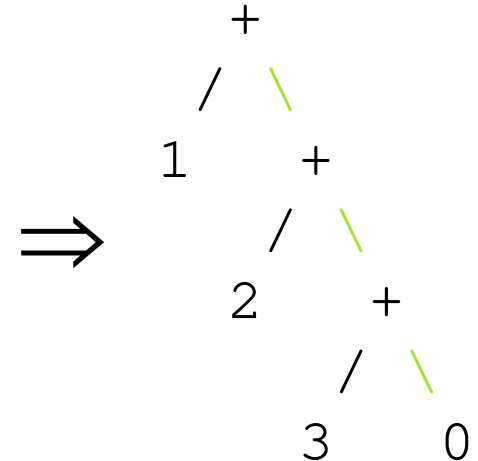
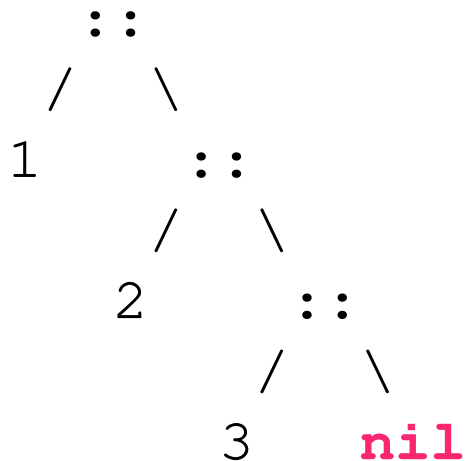
FOLD — SUM

[1;2;3] --- sum ---> 6 (f = (+), b = 0)



FOLD — SUM

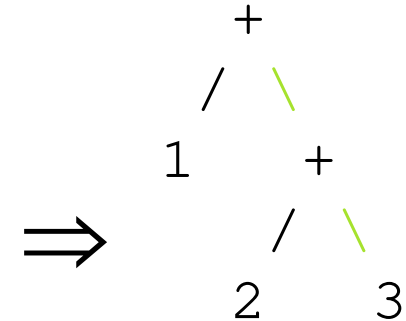
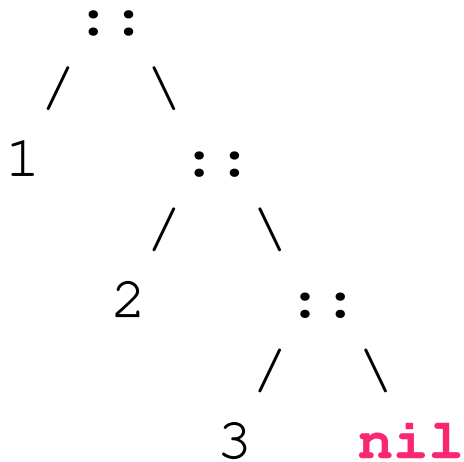
`[1;2;3]` --- `sum` ---> `6` ($f = (+)$, $b = 0$)



$$= 1 + (2 + (3 + 0)) = 6$$

FOLD — SUM

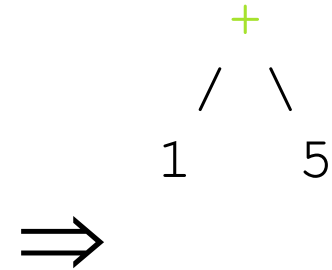
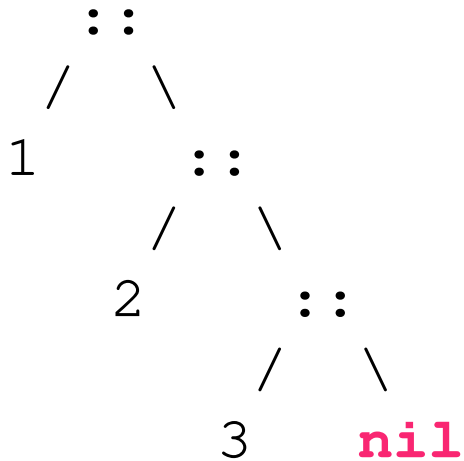
[1;2;3] --- sum ---> 6 (f = (+), b = 0)



$$= 1 + (2 + (3 + 0)) = 6$$

FOLD — SUM

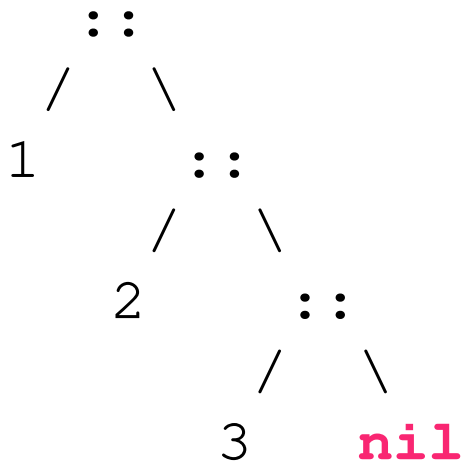
`[1;2;3]` --- `sum` ---> `6` ($f = (+)$, $b = 0$)



$$= 1 + (2 + (3 + 0)) = 6$$

FOLD — SUM

[1;2;3] --- sum ---> 6 (f = (+), b = 0)



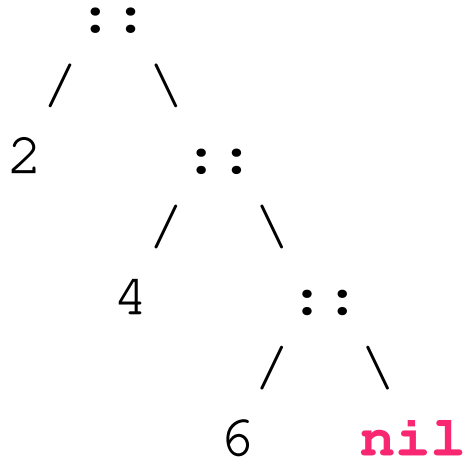
⇒

$$= 1 + (2 + (3 + 0)) = 6$$

FOLD — ALL_EVEN

`[2;4;6] --- all_even ---> true`

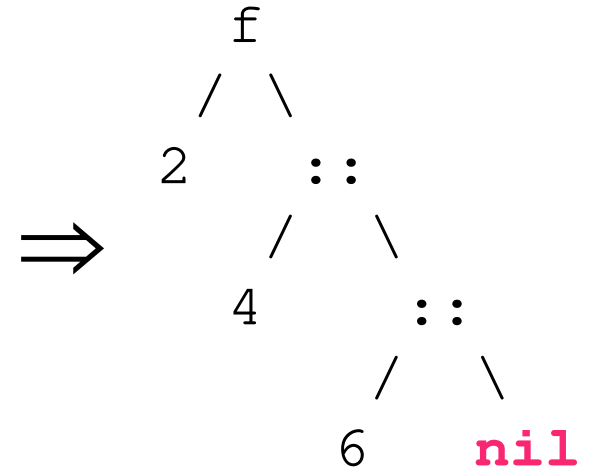
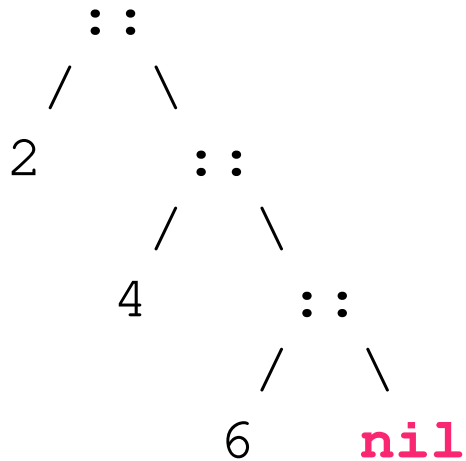
`f = fun head old => old && (even head), b = true`



FOLD — ALL_EVEN

[2;4;6] --- all_even ---> true

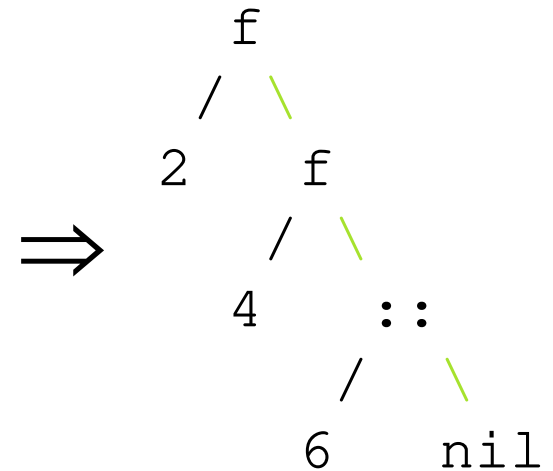
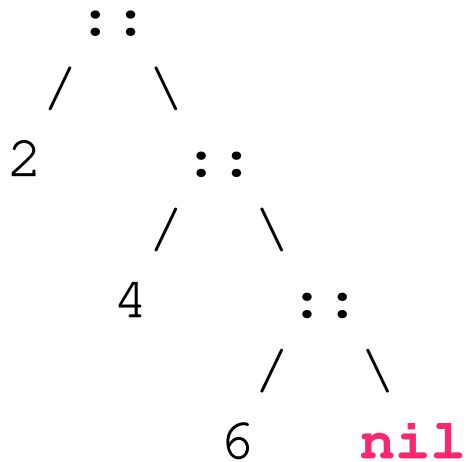
f = fun head old => old && (even head), b = true



FOLD — ALL_EVEN

`[2;4;6] --- all_even ---> true`

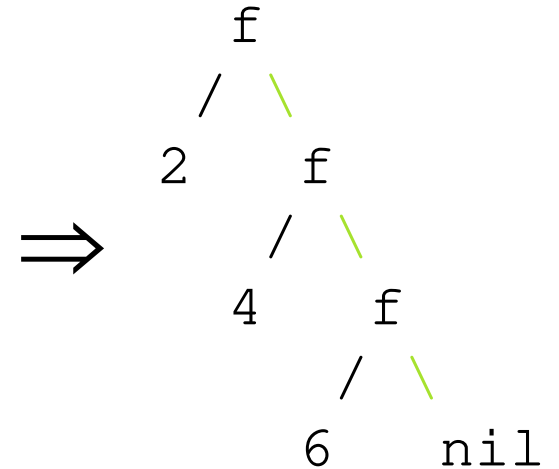
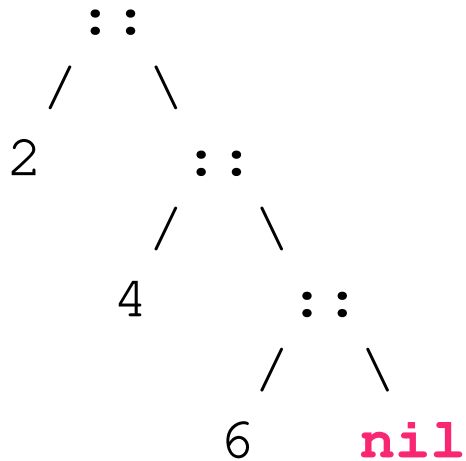
`f = fun head old => old && (even head), b = true`



FOLD — ALL_EVEN

[2;4;6] --- all_even ---> true

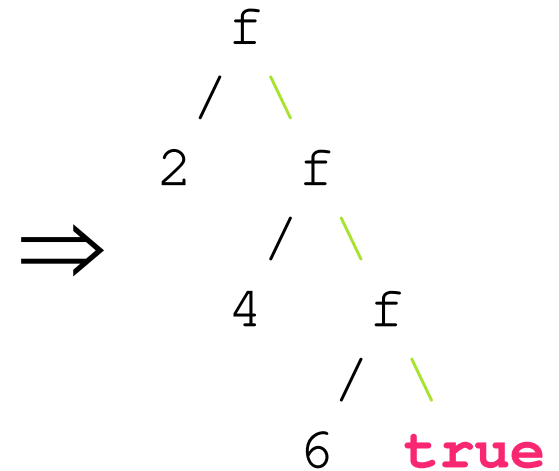
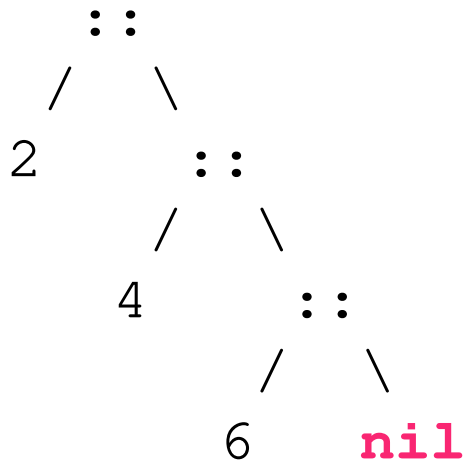
f = fun head old => old && (even head), b = true



FOLD — ALL_EVEN

[2;4;6] --- all_even ---> true

f = fun head old => old && (even head), b = true

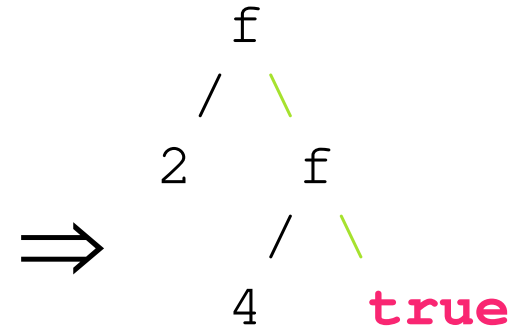
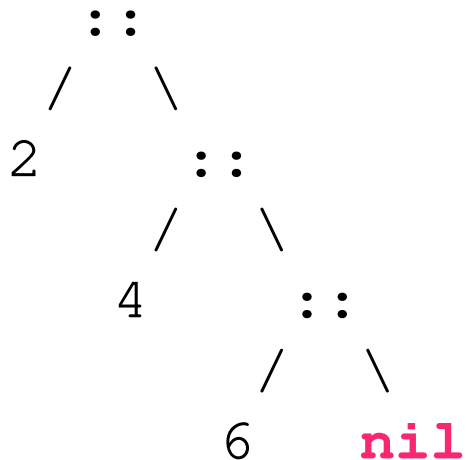


= f 2 (f 4 (f 6 true)) = true

FOLD — ALL_EVEN

[2;4;6] --- all_even ---> true

f = fun head old => old && (even head), b = true

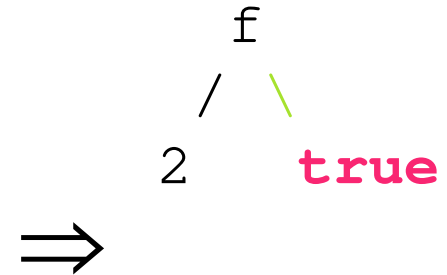
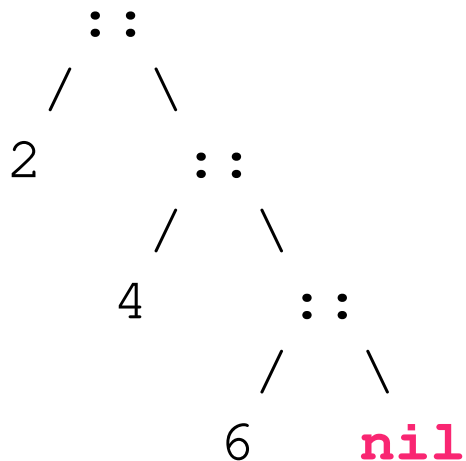


= f 2 (f 4 (f 6 true)) = true

FOLD — ALL_EVEN

[2;4;6] --- all_even ---> true

f = fun head old => old && (even head), b = true



= f 2 (f 4 (f 6 true)) = true

FOLD — ALL_EVEN

[2;4;6] --- all_even ---> true

f = fun head old => old && (even head), b = true



= f 2 (f 4 (f 6 true)) = true

FOLD — ALL_EVEN

`[2;4;6] --- all_even ---> true`

`f = fun head old => old && (even head), b = true`



`= f 2 (f 4 (f 6 true)) = true`

Check `f : nat -> bool -> bool`.

FOLD

```
Fixpoint fold {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)
  : Y :=
  match l with
  | nil => b
  | h :: t =>
  end.
```

FOLD

```
Fixpoint fold {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)
    : Y :=
  match l with
  | nil => b
  | h :: t => f h (fold f t b)
  end.
```

FOLD

```
Check (fold andb) : list bool -> bool -> bool.
```

```
Example fold_example1 :  
  fold andb [true;true;false;true] true = false.
```

```
Example fold_example2 :  
  fold mult [1;2;3;4] 1 = 24.
```

```
Example fold_example3 :  
  fold app [[1];[];[2;3];[4]] [] = [1;2;3;4].
```

FOLDL

```
Fixpoint foldl {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)
    : Y :=
  match l with
  | nil => b
  | h :: t => foldl f t (f h b)
  end.
```

```
Example fold_example1 :
foldl andb [true;true;false;true] true = false.
```

```
Example fold_example2 :
foldl mult [1;2;3;4] 1 = 24.
```

```
Example fold_example3 :
foldl app [[1];[];[2;3];[4]] [] = (* ... *).
```

FOLDL

```
Fixpoint foldl {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)
    : Y :=
  match l with
  | nil => b
  | h :: t => foldl f t (f h b)
  end.
```

```
Example fold_example1 :
foldl andb [true;true;false;true] true = false.
```

```
Example fold_example2 :
foldl mult [1;2;3;4] 1 = 24.
```

```
Example fold_example3 :
foldl app [[1];[];[2;3];[4]] [] = [4;2;3;1].
```

FUNCTIONS THAT CONSTRUCT FUNCTIONS

```
Definition constfun {X: Type} (x: X) : nat -> X :=  
  fun (k:nat) => x.
```

```
Definition ftrue := constfun true.
```


Polymorphism

Higher-Order-Functions

Fold, Map, Filter

Functions that construct Functions

Polymorphism

```
forall X, X -> list X
```

Higher-Order-Functions

Fold, Map, Filter

Functions that construct Functions

Polymorphism

```
forall X, X -> list X
```

Higher-Order-Functions

```
Fixpoint fold {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)  
  : Y := ...
```

Fold, Map, Filter

Functions that construct Functions

Polymorphism

```
forall X, X -> list X
```

Higher-Order-Functions

```
Fixpoint fold {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)  
  : Y := ...
```

Fold, Map, Filter

```
Fixpoint filter {X:Type} (test: X->bool) (l:list X) :list X :=
```

Functions that construct Functions

Polymorphism

```
forall X, X -> list X
```

Higher-Order-Functions

```
Fixpoint fold {X Y: Type} (f : X->Y->Y) (l : list X) (b : Y)  
  : Y := ...
```

Fold, Map, Filter

```
Fixpoint filter {X:Type} (test: X->bool) (l:list X) :list X :=
```

Functions that construct Functions

```
Definition constfun {X: Type} (x: X) : nat -> X :=  
  fun (k:nat) => x.
```